Baseline Reproduction - Technical Memo

Team 11: LiveLaughLoveMachines (LLLM): CSCI 3052U

Date: October 19, 2026
Subject: Baseline Reproduction - Bitcoin Money-Laundering Detection (Elliptic Dataset)

1. Repository Submission

- **Repository URL:** https://github.com/muhammadniazi-dot/CSCI3052U-Project
- **Release Tag:** v1.0-baseline
- **Direct link:** https://github.com/muhammadniazi-dot/CSCI3052U-Project/releases/tag/v1.0-baseline
- **Data pipeline:** scripts/clean_data.py
- **Baseline models and evaluation:** notebooks/02_baseline.ipynb
- **Exploratory data analysis:** notebooks/01_EDA.ipynb
- **Saved metrics:** results/baseline_metrics.json

2. Reproducible Environment & Repository Setup

The codebase for this milestone is tagged v1.0-baseline in the main repo. Reproducibility is ensured via a requirements.txt file (Python 3.13.7) in the root directory which can be used to recreate the environment with pip install -r requirements.txt. All stochastic processes (model initialisation, DummyClassifier state, Decision Tree splits) are fixed via the use of RANDOM_SEED = 42. The Elliptic Bitcoin dataset is downloaded directly from Kaggle (CC BY-NC-ND 4.0) and is not committed to the repo as per the license terms.

3. Working Data Pipeline

The data pipeline is modularized in scripts/clean_data.py and notebooks/02_baseline.ipynb. It executes the following steps:

- **Ingestion:** Reads elliptic_txs_features.csv, elliptic_txs_classes.csv, and elliptic_txs_edgelist.csv into data from the local data/ directory.
- **Cleaning:** Joins files based on txId column, renames feature columns to feat_1 to feat_165 and recodes labels where 1 = illicit, 0 = licit and -1 = unknown.
- **Integrity checking:** Ensures there are no duplicates in txIds, no nulls in feature values, no self-loops in edges and every edge has a valid txId.
- **Splitting:** Does a strict time-based split (time steps 1-34 as training and 35-49 as testing to avoid temporal leakage). Unknown label nodes (~77%) remain in the graph but will not be used in supervised learning via a binary mask.

- **Preprocessing:** Applies StandardScaler to all 166 numerical features without any leakage to the test set.

4. Baseline Methodology

To establish a foundation for future model development, two baseline approaches were evaluated.

- **Naive Benchmark:** A Majority Class Classifier
    - (DummyClassifier(strategy="most_frequent", random_state=42)). Since ~93% of labeled transactions in the test window are licit, this model predicts “Licit” for every instance, exposing the misleading nature of accuracy on imbalanced data.
- **Course-Level Baseline:** A Decision Tree Classifier
    - (DecisionTreeClassifier(max_depth=5, class_weight="balanced", random_state=42)), representing an interpretable, rule-focused classification approach covered in Module 3 of the course. The class_weight="balanced" setting weights the minority (illicit) class higher to address the severe class imbalance, and max_depth=5 prevents overfitting on the training partition.

5. Baseline Table

Models were evaluated on the labeled transactions in time steps 35-49. Due to the severe class imbalance (~6.5% illicit in the test window), F1-Score and Recall on the illicit class are prioritized over raw accuracy.

Model	**Accuracy**	**Precision**	**Recall**	**F1**
Naive Benchmark	0.935033	0.000000	0.000000	0.000000
Decision Tree	0.920336	0.434033	0.744229	0.548299

6. Discrepancy Analysis

Weber et al. (2019) give the classical baseline results for the Elliptic illicit class, the F1-scores ranging from 0.44 (in the case of Logistic Regression) to 0.78 (for Random Forest), according to the type of model. Alarab et al. (2020) also report values similar to these, between 0.44 and 0.56, for comparable tree-based methods.

- **Match:** The accuracy of our naive benchmark of 93.5% corresponds to the expected distribution of classes in the test period, indicating that the pipeline generates an output in accordance with the labeled sample. The 0.000 F1 value of the naive benchmark provides concrete proof of why accuracy is misleading when dealing with highly unbalanced datasets.
- Our Decision Tree F1 value (0.548) easily fits within the range published for tree-based classifiers on the Elliptic dataset.

- There is great harmony between recall (0.744) and precision (0.434) in this model; hence it is able to capture all fraud transactions without having too many false positives. The combination of class_weight = "balanced" and max_depth = 5 ensures that there is no overfitting despite the extreme class imbalance.
- Next steps will involve adjusting max_depth and min_samples_leaf using cross-validation. Random Forest, HistGradientBoosting, GCN, and GAT models will be applied next to build on top of the current feature only model by leveraging the transaction graph.

7. Start / Stop / Continue Plan

Reflecting on the development and evaluation of baseline models, the following adjustments will be made for the next milestones.

- **Start:** Implementing cross-validation to evaluate model configurations more systematically and tune Decision Tree parameters, including max_depth and min_samples_leaf; this will improve generalization and identify a balance between precision and recall.
- **Stop:** Specifying exact versions of the packages (for example, pandas==2.1.0) in requirements.txt; the use of such strict version constraints caused problems when setting up the environment for Python 3.13 because the necessary wheels were missing, which meant a fix had to be introduced halfway through the milestone. From now on, looser constraints (such as pandas>=2.2.0) will be used.
- **Continue:** Writing modular and well-documented preprocessing helpers in the scripts/ directory, incorporating built-in data integrity checks (to prevent duplicate txIds, missing features, and self-loops), and maintaining all random seeds at 42 to ensure cross-run reproducibility.